DAY 20 | HANDS-ON ACTIVITIES

# Git, CI/CD and Docker: Problem Statements

Seven problems. Git and the security scanners run in Colab and VS Code. Docker commands need Docker Desktop on your laptop.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

**How to work:** read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

## Problem 1: Feature Branch and Merge

**Level:** Warm-up | **Concepts:** init, add, commit, switch, merge

Create a repo for your Task API, make two commits, create feature/health-check, add a /health endpoint, commit, merge into main, and delete the branch.

### Sample run / expected result

```
Fast-forward
 app/main.py | 4 ++++
Deleted branch feature/health-check
b2c3d4e Add health check endpoint
a1b2c3d Initial FastAPI app
```

### Hints

- git config user.name and user.email first.
- git branch -d feature/health-check

## Problem 2: Resolve a Conflict

**Level:** Easy | **Concepts:** merge conflicts

Change the same line on two branches, merge them, resolve the conflict by keeping a combined version, and commit.

### Sample run / expected result

```
CONFLICT (content): Merge conflict in greeting.py
... after fix ...
[main 7c1d2e3] Resolve greeting conflict
```

### Hints

- Search the file for <<<<<<< and >>>>>>> markers.

## Problem 3: Make the Security Gate Fail NEW (v4)

**Level:** Medium | **Concepts:** bandit, gates, exit codes

Run bandit on the deliberately unsafe app in the notebook. Write gate(findings) that returns exit code 1 if any finding is medium or high. Fix each finding, re-run, and show the gate pass.

### Sample run / expected result

```
Run 1: 5 findings, 3 blocking (HIGH 2, MEDIUM 1) -> gate FAIL
Run 2: 3 LOW, 0 blocking -> gate PASS
```

### Hints

- bandit -f json gives machine-readable output.
- Fix the code; do not just add # nosec.

## Problem 4: Secret Scanner NEW (v4)

**Level:** Medium | **Concepts:** regex, Git history

Write scan_secrets(path) that flags API key and password patterns in files, and run it on a repo where a key was committed and then deleted. Show that the key is still in git log -p.

### Sample run / expected result

```
config.py:3  possible API key
history: commit 4f2a1c0 adds API_KEY=... (still recoverable)
```

### Hints

- Revoke and rotate first. Cleaning history comes second.

## Problem 5: Vulnerability Report v2 NEW (v4)

**Level:** Medium | **Concepts:** triage, remediation, evidence

Fill the vulnerability report template for every finding from bandit, pip-audit and the secret scan: ID, finding, tool, severity, file, proposed fix, fixed by (human or Codex-assisted), verified, evidence. Include scan output before and after the fixes.

### Sample run / expected result

```
| V-01 | shell=True in run_report | bandit | High | app/main.py | list args | Codex-assisted
```

### Hints

- Use shared/templates/Vulnerability_Report_Template.md.
- pip-audit needs internet; run it in Colab or VS Code.

## Problem 6: Codex as Reviewer NEW (v4)

**Level:** Challenging | **Concepts:** AI pull request review, human approval

Open a pull request with one deliberate bug and one missing test. Ask Codex in VS Code to review the diff using the review prompt in the notebook. Record each comment as accepted or rejected with a reason, then approve or request changes yourself.

### Sample run / expected result

```
Comment 1: missing test for empty title  -> accepted, test added
Comment 2: rename variable x            -> rejected, out of scope
Decision: approved by human reviewer
```

### Hints

- Paste only the diff, never secrets or customer data.

## Problem 7: Smaller and Safer Image NEW (v4)

**Level:** Stretch | **Concepts:** slim images, non-root user, scanning

Write a hardened Dockerfile for the Task API and a multi-stage Dockerfile for the React app. Compare docker images sizes before and after, confirm the container runs as a non-root user, and scan the image.

### Sample run / expected result

```
tasks-api:1.0 1.1GB (root)
tasks-api:1.1 190MB (uid 10001)
image scan: 0 critical, 0 high
```

### Hints

- docker run --rm tasks-api:1.1 id
- Image scanners include Trivy and Docker Scout.